Simulation Report

Functional and Cycle-Level Simulation of the
FPGA Matrix-Multiplication Accelerator

TeraNova – FPGA Acceleration

Author	Kai Janssen
Student number	4994817

Date	29 October 2026, updated 30 October 2026
Platform	AMD ZCU102 (Zynq UltraScale+ `xczu9eg-ffvb1156-2-e`)
Tools	GHDL 4.1.0; Vitis HLS 2025.1.1 (C simulation, C/RTL co-simulation)

All results in this report come from simulation. None of them has been measured on the board yet.

Abstract

While the ZCU102 board is not available, I simulated every part of the accelerator that can be simulated without it. This covers the complete programmable-logic (PL) data path, from the block RAMs to the result memory, including the real RTL that Vitis HLS generated for `RAM_INTERFACE`. All 96 simulation runs produce the correct 50×50 result: 94 GHDL runs (two data sets, a range of writer speeds and memory latencies, a restart without reset, three partitioner versions) and the two Vitis HLS simulations from 7 October. With realistic memory behaviour, one multiplication takes **101 186 clock cycles (1.012 ms at 100 MHz)** on the PL. Handshake monitors confirm the main finding of the earlier analysis with exact numbers. The partitioner hands over one block pair every 101 cycles, and 50 of those cycles are wait states that do no work. The multiplier is busy only 5 000 of 101 008 cycles (4.95 %). The simulation also found one thing the analysis had not: the generated `RAM_INTERFACE` waits for the write response after every 5-element burst. In steady state it needs $85 + 5L$ cycles per output block, where L is the memory's write-response latency. That is harmless today, but it becomes the bottleneck once the partitioner is made faster. I simulated two faster partitioners as the first improvement. Removing the wait states halves the PL run time to 51 186 cycles. A pipelined version that reads one element per cycle brings it to **27 186 cycles (0.272 ms, 3.7× faster)**. That version only reaches its speed if the memory confirms a write within 37 cycles; above that, the result writer sets the pace. The CPU side (loading the matrices and reading the result back) cannot be simulated and is left for the board measurement.

Update 30 October. The improvements are now simulated as an *improvement ladder* (Section 9): every change is added on top of the previous one, in the order in which it will be built, and the whole system is simulated again after each step. Four steps take the PL run time from 101 186 to **10 073 cycles (0.101 ms, 10.0×)**: the pipelined partitioner (27 186), a result writer that no longer waits for each write response (27 071, and insensitive to the memory latency up to about 215 cycles), a partitioner that copies A and B on chip once and reuses them ten times (10 052), and a burst data path in which the PL reads A and B from the processor's DDR itself (10 073). The last step hardly changes the PL time; its gain is on the CPU side, which no longer has to write 10 000 values into block RAMs one at a time. Every step passes all functional tests.

Contents

1 What can be simulated, and what cannot **3**

2 Simulation setup **3**

3 Functional results **5**

4 Where the clock cycles go **6**

5 The real result writer **7**

6 HLS synthesis results **8**

7 Prediction for the board baseline **9**

8 Simulated first improvement: a faster partitioner **9**

9 The improvement ladder **11**

10 Limitations **14**

11 Conclusion **15**

A Reproducing the results **15**

B Files **15**

1 What can be simulated, and what cannot

Table 1 lists each part of the system and whether simulation can give a reliable answer for it. The rule is simple: anything inside the PL is described by VHDL or HLS code that can be simulated cycle by cycle. The ARM processor, the PS–PL interconnect and the DDR4 controller are not, so their timing has to be measured on the board.

Part	**How**	**Simulated?**
`matrix_partitioner_B` (rev. 4)	Its VHDL, in GHDL	Yes, cycle-exact
`MatMul5x5_PAR_IF` (multiplier)	Its VHDL, in GHDL	Yes, cycle-exact
`MatMul5x5_Accumulator`	Its VHDL, in GHDL	Yes, cycle-exact
`RAM_INTERFACE` (result writer)	HLS C simulation; xsim co-simulation; generated RTL in GHDL	Yes, cycle-exact
Result memory (PL DDR4 via MIG)	AXI4 memory model with adjustable latency	Behaviour only; latency is a parameter
Block RAMs (`Part0`–`Part3`)	Behavioural model, read latency 1	Yes (as configured in the design)
CPU loading A and B (10 000 AXI writes)	–	No: needs the board
CPU reading the result (2 500 AXI reads)	–	No: needs the board
Interrupt path `done` → GIC	–	No: needs the board
Timing closure at 100 MHz	–	No: needs the licensed Vivado build

Table 1: Simulation scope.

2 Simulation setup

2.1 Testbenches

Most of the testbenches already existed from the IP restoration in October. For this report I added a performance version of the chain testbench and a version that uses the real HLS RTL. The design files themselves were not changed.

The monitors in `tb_chain_perf` only read signals; they do not drive anything. This means the cycle counts are those of the unmodified design. Every testbench run also checks the complete result against a reference computed in Python, so a performance number is only reported for a run that produced the correct result.

2.2 Test data

Two data sets are used:

- **`helloworld` data:** $A_{ij} = 50i + j + 1$, $B_{ij} = 50i + j + 2$, the same values as the board application;

Testbench	What it does	Status
`RAM_INTERFACE_TEST.cpp`	HLS C testbench: feeds 2 complete matrices block by block, checks every word in memory and the `done` behaviour	Existing (2 Oct)
`tb_chain.vhd` + `run.py`	Complete PL data path with a model of the writer handshake; compares the 50×50 result with Python	Existing (2 Oct)
`tb_chain_perf.vhd` + `run_perf.py`	Same as `tb_chain`, plus passive monitors on every valid/ready handshake that count cycles per stage	New
`tb_chain_rtl.vhd` + `run_rtl.py`	Data path with the *real* `RAM_INTERFACE` RTL from Vitis HLS, writing through `m_axi` into an AXI4 memory model; ends when `done` rises. `RUNS=2`: a second complete run without reset (restart test)	New
`axi_mem_model.vhd`	AXI4 write slave: up to 4 outstanding addresses, byte strobes, write response `B_LATENCY` cycles after `WLAST`, optional throttling of `WREADY`; counts bursts and beats	New
`run_all.py`	Compiles everything and runs the 31 GHDL cases for the current design; writes `results.json`	New
`run_variants.py`	Runs the testbenches for each partitioner version (rev. 4, v5a, v5b) in its own build folder (63 runs); writes `variants_results.json`	New

Table 2: Testbenches used in this report (all in `C:\PLD\sim`, HLS in `C:\PLD\hls\RAM_INTERFACE`).

- **random 64-bit data:** full-range random values (fixed seed). This checks the upper bits, the wrap-around modulo 2^{64} and the signed/unsigned mismatch noted as finding F9 in the analysis.

2.3 Clock and units

All PL logic runs on one 100 MHz clock in the design, so one cycle is 10 ns. Cycle counts start at the start pulse.

2.4 The generated RTL in GHDL

`tb_chain_rtl` uses the seven VHDL files that Vitis HLS generated on 7 October (`work\hls\impl\vhdl`), i.e. exactly the RTL inside the packaged IP. GHDL needed one change: line 539 of `RAM_INTERFACE_gmem_m_axi.vhd` uses a VHDL-2019 conditional constant,

which GHDL's VHDL-2008 mode does not accept. In the simulation copy (`sim\hls_rtl`) that line is replaced by an equivalent function call, marked `SIM PATCH`. It selects the same value. The packaged IP is not affected.

3 Functional results

All simulation runs of the current design produce the correct result (Table 3). The HLS C simulation, synthesis and xsim co-simulation are from 7 October (logs in `hls\RAM_INTERFACE`); all GHDL runs were done today from a clean folder with `run_all.py` and `run_variants.py`. The partitioner versions of Section 8 pass the same tests.

Test	Condition	Result
HLS C simulation, `RAM_INTERFACE`	2 matrices back to back, full 64-bit values	**PASS**
HLS C synthesis + packaging	`xczu9eg`, 100 MHz target	**PASS**
C/RTL co-simulation (xsim, `ap_ctrl_hs` variant)	2×100 blocks	**PASS**
`tb_chain`, writer model	`helloworld` data and random 64-bit data	**PASS** (2 runs)
`tb_chain`, slow writer	1 500 cycles per block, both data sets	**PASS** (2 runs)
`tb_chain_perf`	both data sets	**PASS** (2 runs)
`tb_chain_perf`, writer sweep	30 to 1 500 cycles per block (11 points)	**PASS** (11 runs)
`tb_chain_rtl`, real `RAM_INTERFACE`	both data sets, $L = 20$ cycles	**PASS** (2 runs)
`tb_chain_rtl`, latency sweep	$L = 1$ to 500 cycles (10 points)	**PASS** (10 runs)
`tb_chain_rtl`, throttled write channel	`WREADY` low every 2nd / 4th cycle	**PASS** (2 runs)
`tb_chain_rtl`, restart	second complete run without reset, both data sets	**PASS** (2 runs)

Table 3: Functional results. "PASS" means all 2 500 result values are correct and, for `tb_chain_rtl`, no AXI protocol assertion fired and `done` rose before the time-out.

Three behaviours that matter for the board run are confirmed:

- **No lost or duplicated blocks under back-pressure.** Even when the writer is 50 % slower than the data path, all 100 output blocks arrive exactly once. This was the weakness of the previous group's `ap_none` handshake (finding F11).
- **`done` rises only after the last write is acknowledged.** In `tb_chain_rtl` it rises 179 cycles after the last block was accepted ($L = 20$), after the AXI write response of the final burst.
- **Restart works without a reset.** In `tb_chain_rtl` with `RUNS=2`, a second start after `done` gives a second correct result in exactly the same time (101 186 cycles); `done` goes low during the second run and rises again at its end. The HLS testbench and co-simulation show the same for the writer alone. This is what the board application relies on when it runs 10 times in a row.

4 Where the clock cycles go

4.1 Per-stage counters

Table 4 shows the monitor results of `tb_chain_perf`. They are identical for both data sets: the run time does not depend on the values.

Measurement	Value	Meaning
Total PL run time	101 008 cycles	1.010 ms at 100 MHz
Block pairs handed to the multiplier	1 000	$10 \times 10 \times 10$
Interval between block pairs	101 cycles (min = max)	partitioner sets the pace
Cycles the partitioner waited for the multiplier	0	the multiplier never blocks
Multiplier busy (not ready)	5 000 cycles	4.95 % of the run
Multiplier latency per block pair	5 cycles (min = max)	accept → product valid
Cycles the multiplier waited for the accumulator	0	
Output blocks to the writer	100	
Interval between output blocks	1 010 cycles	10×101
First output block	cycle 1 018	
Cycles the accumulator waited for the writer	0	with a 30-cycle writer model

Table 4: Handshake monitor results, `tb_chain_perf`, writer model busy 30 cycles per block.

The total follows exactly from the partitioner's state machine:

$$T_{\mathrm{PL}} = \underbrace{1000}_{\text{block pairs}} \times \Big(\underbrace{25 \times 4}_{\text{elements}\times\text{states}} + \underbrace{1}_{\text{handshake}} \Big) + \underbrace{2}_{\text{start}} + \underbrace{5}_{\text{multiplier}} + \underbrace{1}_{\text{accumulator}} = 101\,008 \text{ cycles}$$

The 4 states per element are `ISSUE_ADDR`, `WAIT1`, `WAIT2` and `CAPTURE`. The last three terms are the latency of the final block: start-pulse detection, the multiply, and the last accumulation.

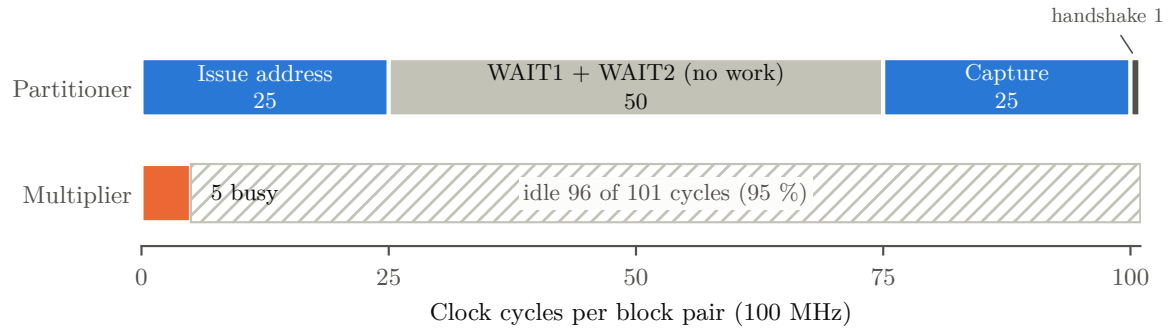


Figure 1: One block pair takes 101 cycles. Half of them are the `WAIT1`/`WAIT2` states, and the multiplier works for only 5.

4.2 What this confirms

Figure 1 turns three estimates from the analysis report into measured facts:

- **F3, unnecessary wait states:** 50 of every 101 cycles (49.5 %) are spent in `WAIT1`/`WAIT2`. The BRAMs in the design (and the model) have a read latency of 1 cycle, so the data is valid one cycle after the address and the two wait states add nothing. Section 8 simulates versions without them.
- **F4, one element per cycle at best:** even without wait states, the 25 elements of a block are read one at a time.
- **F7, oversized multiplier:** the single-cycle 5×5 multiplier, which uses 1 257 DSPs, is idle 95 % of the time. The analysis estimated ∼5 cycles per block pair and ∼95 % idle; the simulation measures exactly 5 and 95.05 %.

5 The real result writer

5.1 Run time with the generated RTL

With the real `RAM_INTERFACE` RTL and a write-response latency of $L = 20$ cycles, the complete PL run, from start until `done`, takes **101 186 cycles (1.012 ms)**. The data path itself is unchanged: all counters of Table 4 are identical. The rest is the writer finishing the last block: `done` rises $79 + 5L$ cycles after the writer accepted it.

Measurement	Value	Note
Start → `done`	101 186 cycles	1.012 ms at 100 MHz
Last block accepted → `done`	$79 + 5L$ cycles	179 cycles at $L = 20$
AXI write bursts	504	500 rows + 4 splits
AXI write beats	2 500	one per result value
Burst length	1 to 5 beats	one row of a 5×5 block
Write bursts in flight (max)	2	the IP is configured for 4
AXI read transactions	0	as intended

Table 5: `tb_chain_rtl`: real `RAM_INTERFACE` RTL, AXI memory model with $L = 20$.

Short bursts (finding F6). The writer forms one burst per row of a block: 5 consecutive addresses. The next row of the same block is 50 elements further on in memory, so it needs a new burst. The 4 extra bursts appear where a row crosses a 4 KB address boundary: the result is 20 000 bytes, so it crosses the boundaries at 4, 8, 12 and 16 KB, and AXI does not allow a burst across them. The `max_write_burst_length=16` setting is never reached.

A wait after every row. The writer's time per block grows by 5 cycles for every extra cycle of memory latency (Figure 2b). The reason is in the generated pipeline (`RAM_INTERFACE_RAM_INTERFACE_Pipeline_ROWS_COLS.vhd`). Vitis HLS put the wait for the write response (`m_axi_gmem_0_BVALID`) inside the pipelined loop, at the end of every row. So the writer stalls 5 times per block, once for each burst, instead of letting the responses overlap. For a fast memory this costs 79 cycles per block. The xsim co-simulation of 7 October measured 780 ns, i.e. 78 cycles, per block, which is consistent. It also explains why the HLS estimate of 41 cycles was too low: HLS schedules with an ideal memory.

5.2 When does the writer become the bottleneck?

The accumulator presents a finished block every 1 010 cycles. As long as the writer needs less than that, it is hidden behind the partitioner. Figure 2 shows both ways this was tested. In (a), the writer model is given a fixed busy time per block; in (b), the real RTL runs against memories with increasing write-response latency. The two agree: the run time stays at the partitioner

limit until the writer needs more than about 1 009 cycles per block. The real RTL keeps up at $L = 185$ and stalls the accumulator from $L = 186$ (1.86 µs). In steady state it needs $85 + 5L$ cycles per block: $79 + 5L$ to write a block, plus 6 cycles to finish the call and accept the next block. This simple rule predicts the limit exactly for all three partitioner versions (Section 8). Above the limit, every extra cycle of latency adds 500 cycles to the run (100 blocks × 5 rows).

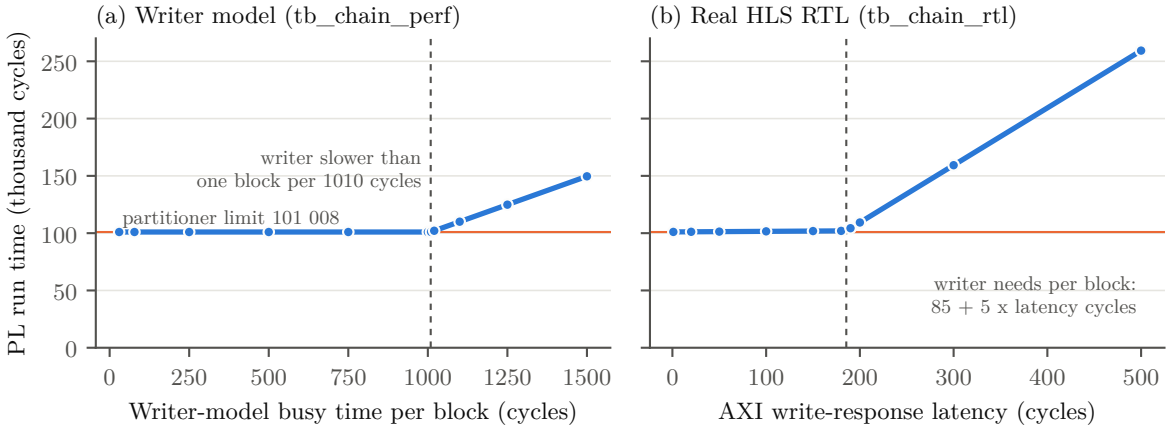


Figure 2: PL run time against writer speed. (a) Writer model with a fixed busy time per block. (b) Real `RAM_INTERFACE` RTL against the AXI memory's write-response latency. The orange line is the partitioner limit (101 008 cycles).

Throttling the write channel has little effect. With `WREADY` low every 2nd or 4th cycle, the run takes 101 211 or 101 194 cycles (+25 and +8). Data beats are not the limit; waiting for write responses is.

5.3 What it means for the board

The latency of the PL DDR4 path (SmartConnect, clock-domain crossing to the 300 MHz MIG, the DDR4 itself) is not known yet. It is very unlikely to be anywhere near 1.86 µs, so with today's partitioner the real writer should not slow the run down. The predicted PL compute time on the board is therefore 1.011 ms ($L = 1$) to 1.013 ms ($L = 50$).

6 HLS synthesis results

For completeness, the synthesis results of the rebuilt `RAM_INTERFACE` (Vitis HLS 2025.1.1, 7 October, `work\hls\syn\report\RAM_INTERFACE_csynth.rpt`):

Item	Result
Estimated clock period (target 10 ns)	7.30 ns ($F_{max} \approx 137$ MHz)
Main loop `ROWS_COLS`	pipelined, II = 1, depth 11
Latency per block (HLS estimate, ideal memory)	41 cycles
Latency per block (co-simulation, xsim)	78 cycles
Latency per block (GHDL, $L = 1$)	79 cycles
Resources	2 708 FF, 1 943 LUT, 2 DSP, 0 BRAM (<1 %)

Table 6: HLS results of `RAM_INTERFACE`.

7 Prediction for the board baseline

Simulation fixes the PL part of the baseline. The end-to-end time on the board is

$$T_{\text{total}} = \underbrace{10\,000 \cdot t_{\text{write}}}_{\text{load A, B}} + \underbrace{T_{\text{PL}}}_{\approx 1.012\text{ ms (simulated)}} + \underbrace{2\,500 \cdot t_{\text{read}}}_{\text{read back C}}$$

Here t_{write} and t_{read} are the times of one CPU access across the PS–PL boundary. The measurement application in `C:\PLD\baseline` measures both directly (its "latency probes"). Table 7 lists what simulation predicts and what the board has to add.

Quantity	**Simulation**	**Board**
PL compute (start → `done`)	1 011–1 013 µs ($L \leq 50$)	to measure; should match
Load A and B (10 000 writes)	not simulatable	to measure
Read-back (2 500 reads)	not simulatable	to measure
CPU-only reference	not simulatable	to measure
Correct result, restart	PASS (2 runs without reset)	to confirm with 10 runs
`done` interrupt	not simulatable	to confirm

Table 7: Simulation prediction and the remaining board measurements.

If the board's PL compute time differs from 1.011–1.013 ms by more than a few microseconds, the hardware does not behave like the simulated RTL. That would point at the build (for example a stale bitstream) rather than the design.

8 Simulated first improvement: a faster partitioner

The analysis report names the partitioner's wait states as the cheapest improvement. I wrote two versions and ran them through exactly the same testbenches as the current design. Both keep the entity name, generics, ports, block order and output packing of revision 4, so they are drop-in replacements (`sim\variants`):

- **v5a, without wait states:** revision 4 with `WAIT1` and `WAIT2` removed; nothing else changed. 2 cycles per element.
- **v5b, pipelined:** a new BRAM address every cycle; each element is captured two clock edges after its address was set (address register, then the BRAM's read latency of 1). 1 cycle per element plus 2 to drain the pipeline.

Both rely on the BRAM read latency being exactly 1 cycle and the address multipliers adding no delay. I checked this in the block design's IP configuration (`.xci` files of `Part0`–`Part3` and `mult_gen_0/1`): all four BRAMs have `C_READ_LATENCY_B = 1` with no output registers, and both multipliers have `C_LATENCY = 0`. The testbench's BRAM model therefore matches the hardware.

8.1 Results

Table 8 shows that both versions do exactly what was expected. The run time follows the cycles per block pair: v5a halves it and v5b takes 27 186 cycles. That is 3.72× faster on the PL side and close to the ∼4× the analysis estimated. All functional tests pass for both versions: both data sets, a writer that is 50 % slower than the data path (back-pressure), and a second run without reset.

	rev. 4 (current)	v5a	v5b
Cycles per block pair	101	51	27
Output block every	1 010 cycles	510 cycles	270 cycles
PL run, writer model (`tb_chain_perf`)	101 008	51 008	27 008
PL run, real `RAM_INTERFACE`, $L = 20$	101 186	51 186	27 186
at 100 MHz	1.012 ms	0.512 ms	0.272 ms
speed-up of the PL part	1.00×	1.98×	3.72×
Multiplier busy	4.95 %	9.80 %	18.51 %
Largest L the writer can hide (simulated)	185 cycles	85 cycles	37 cycles
Functional tests (both data sets, back-pressure, restart)	**PASS**	**PASS**	**PASS**

Table 8: The three partitioner versions in the same testbenches (`run_variants.py`). Cycle counts are identical for `helloworld` and random 64-bit data.

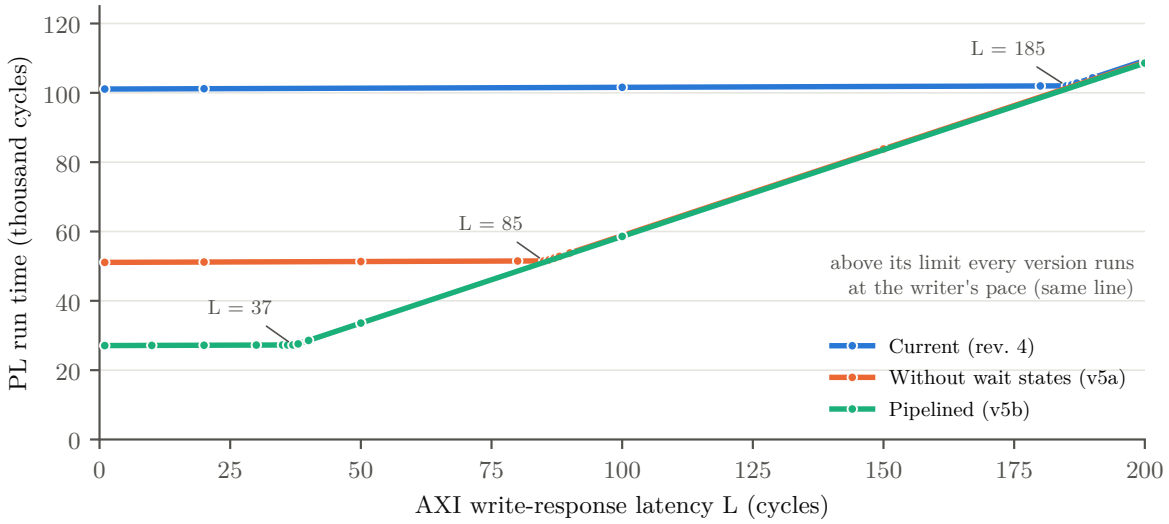


Figure 3: PL run time of the three partitioner versions against the write-response latency of the result memory (real `RAM_INTERFACE` RTL). Each version is flat until the writer can no longer keep up; above that point all versions run at the writer's pace.

8.2 The writer becomes the limit

Figure 3 confirms the prediction of Section 5.2. A faster partitioner delivers finished blocks more often, so the writer has less time for each. With v5b it gets 270 cycles per block, so it can only hide a write-response latency of up to 37 cycles (370 ns at 100 MHz); from 38 cycles on it stalls the accumulator. Above its limit each version runs on the same line: the writer, not the partitioner, then sets the speed, and a faster partitioner gains nothing.

A latency of a few hundred nanoseconds is within the range a DDR4 path through a SmartConnect and a clock-domain crossing can have. Whether v5b reaches its full 3.7× on the board therefore depends on a latency that is not measured yet. Two follow-ups make it independent of that latency:

- change `RAM_INTERFACE` so that it no longer waits for each row's write response before the next row (overlapping responses), or collect a full result row and write it as one long burst;
- check the result in `tb_chain_rtl`: the writer's limit should move far to the right in

Figure 3.

8.3 What is needed to build it

1. Copy `sim\variants\matrix_partitioner_B_v5b.vhd` over the partitioner source in `ip_repo\matrix_partitioner_B` (keep the old file in `backup`).
2. Re-package it with core revision 5 (copy of `1_repackage_partitioner.tcl`) and upgrade it in the block design; the ports are unchanged, so no rewiring is needed.
3. Badr builds the bitstream. Timing at 100 MHz should be checked: v5b computes a new address every cycle with the same address arithmetic that revision 4 already uses.
4. Run the measurement application on the board and compare the PL compute time with 0.272 ms.

I recommend building v5b rather than v5a. It costs the same build effort and gives almost twice the gain of v5a.

Update 30 October: this has been done. v5b is core revision 5 of `matrix_partitioner_B` on the git branch `partitioner-v5b`, rung 1 of the improvement ladder (Section 9). The writer change proposed above is rung 2.

9 The improvement ladder

9.1 Method

The board can only show the effect of a change if the changes are built and measured one at a time. The improvements are therefore organised as a *ladder*: each rung adds one change on top of all earlier rungs, in the order in which they will be built. Every rung is a separate git branch (Table 9). The branch `main` stays at the baseline (tag `baseline-1`) until all rungs have been measured.

After every change the complete PL data path is simulated again with `run_ladder.py`. For each rung it runs four cases with the real RTL of every block: the `helloworld` data and random 64-bit data at the nominal write-response latency $L = 20$ cycles, a second complete run without reset, and a stress case with $L = 100$ cycles. Every case is compared with the exact 50×50 product. The rungs and their source files are listed in `ladder.json`; the results are written to `ladder_results.json` and `ladder.md`.

9.2 Results

Rung	Change (git ref)	$L = 20$	vs prev.	$L = 100$	Tests
0	Baseline: restored design (`baseline-1`)	101 186	–	101 586	**PASS**
1	Partitioner v5b, pipelined fetch (`partitioner-v5b`)	27 186	3.72×	58 576	**PASS**
2	Writer with overlapped write responses (`writer-fix`)	27 071	1.00×	27 151	**PASS**
3	Partitioner v6: on-chip copy, 10× reuse (`partitioner-v6`)	10 052	2.69×	18 052	**PASS**
4	Burst data path via `S_AXI_HP0` (`burst-datapath`)	10 073	1.00×	18 153	**PASS**
	Rung 4 against the baseline	**10.0×**		**5.6×**	

Table 9: PL run time (start to `done`, clock cycles at 100 MHz) after every rung. Each rung contains all earlier ones. The cycle counts are identical for both data sets and for the second run without reset.

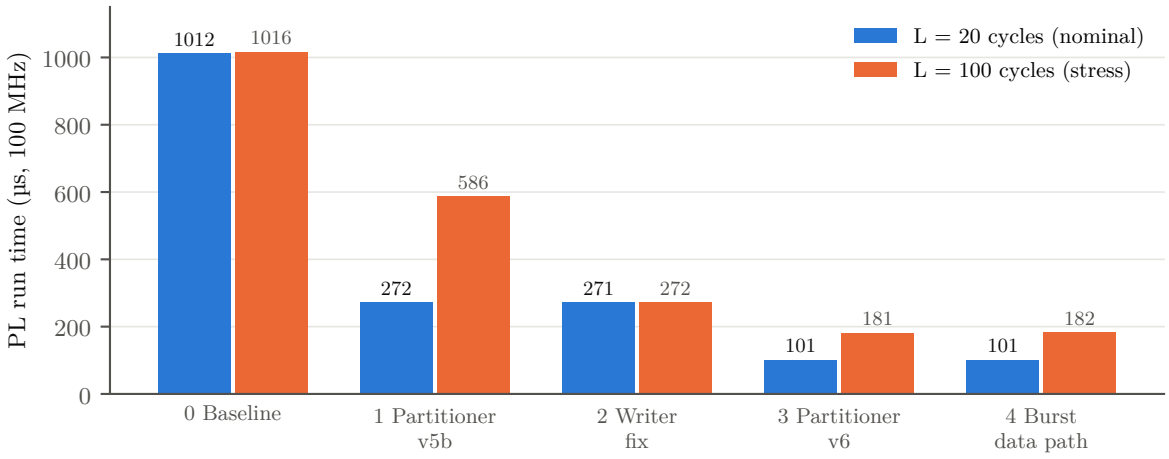


Figure 4: PL run time per rung at the nominal and the stress write-response latency. Rung 1 is the only rung whose speed depends strongly on the latency: its writer cannot keep up above $L = 37$. Rung 2 removes that dependence.

9.3 Rung 2: the result writer

Section 8 showed that after rung 1 the writer sets the pace as soon as $L > 37$ cycles. The generated `RAM_INTERFACE` waited for the write response of every 5-element row before it started the next row. The new version (branch `writer-fix`) uses an explicit burst interface (`hls::burst_maxi`). It first issues the five write requests of a block, then sends the 25 values, and only then collects the five responses, with up to 8 writes in flight. It now needs about $L + 52$ cycles per output block instead of $85 + 5L$, so at rung 1's block interval of 270 cycles it keeps up until $L \approx 215$ cycles. The nominal run gains only 115 cycles, but the stress run drops from 58 576 to 27 151 cycles (2.16×). The speed of rung 1 therefore no longer depends on a latency that has not been measured yet. The new RTL was generated by Vitis HLS and co-simulated (PASS) before it was copied into the ladder. The monitors see up to 6 writes in flight.

9.4 Rung 3: on-chip copy and reuse

Up to rung 2 every block of A and B is read 10 times from the external block RAMs, one element per cycle (finding F5 of the analysis report). Partitioner v6 first copies A and B *once* into 25 internal memory banks per matrix (2 500 cycles), arranged so that the 25 elements of a block are in 25 different banks. A whole block of A and of B can then be read in one cycle. The first block pair leaves the partitioner after 2 508 cycles; from then on a pair can be ready every 6 cycles, the multiplier's pace. The run drops to 10 052 cycles (2.69×).

The handshake monitors show that the writer is the limit again. It takes one output block every 74–75 cycles ($\approx L + 54$), while the multiplier and accumulator could deliver one every 60. The partitioner waits 3 460 cycles in total for the multiplier, and the accumulator waits 1 873 cycles for the writer. At $L = 100$ the block interval grows to 154–155 cycles, which is why the stress run takes 18 052 cycles.

9.5 Rung 4: burst data path

Up to rung 3 the CPU writes A and B into four block RAMs, as 10 000 single 32-bit writes across the PS–PL boundary, and reads the 2 500 results back from the PL DDR4 one at a time (finding F1). In rung 4, A, B and C are in the processor's own DDR. Partitioner v7 fetches A and B itself through an AXI4 read master: 128-bit beats (two elements each), bursts of at most 256 beats that never cross a 4 KB boundary, and up to 8 bursts in flight. A and B take 10 bursts together. `RAM_INTERFACE` writes C to the same memory, and both reach it through the high-performance port `S_AXI_HP0_FPD`.

For this rung the testbench (`tb_chain_axi.vhd`) has a second memory model, `axi_rd_mem_model.vhd`. It is an AXI4 read slave with a configurable read latency, and it also checks the protocol rules (16-byte beats, INCR bursts, alignment, no burst across 4 KB). Because a beat carries two elements, loading the 2 500 beats takes as long as loading 2 500 element pairs from block RAM in rung 3. The PL run time is therefore almost unchanged: 10 073 cycles (+21) at $L = 20$ and 18 153 (+101) at $L = 100$. The read latency is paid once per burst, not once per element.

9.6 Estimated end-to-end time

What the user waits for is the PL run time *plus* the CPU's work to move the data in and out. That CPU work cannot be simulated, so Figure 5 uses estimates (`cpu_estimates` in `ladder.json`). For rungs 0–3 these are 150 ns per BRAM write and 300 ns per PL-DDR4 read, the midpoints of the analysis report. For rung 4 they are 50 ns per 64-byte cache line that is flushed or invalidated, plus 20 ns per cached read. The estimates will be replaced by the latency probes and cache timings that the measurement applications print on the board.

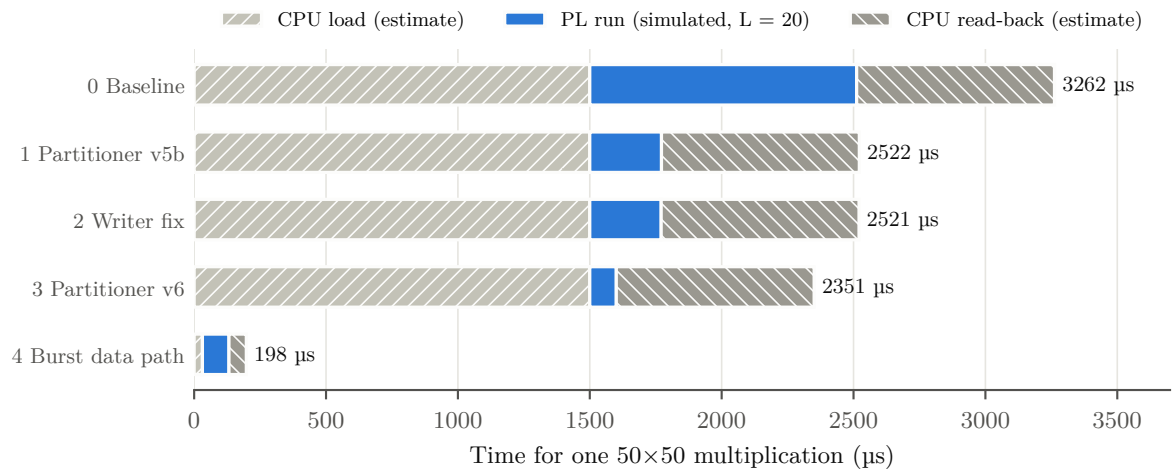


Figure 5: Estimated time for one multiplication including the CPU's data movement. Hatched parts are estimates, blue parts are simulated. Rungs 1–3 only shorten the PL part; rung 4 removes most of the CPU part.

With these estimates, rungs 1–3 only reduce the end-to-end time from about 3.3 ms to 2.4 ms, because the CPU's 10 000 BRAM writes and 2 500 slow reads remain. Rung 4 brings it to about 0.2 ms, 16× less than the baseline. That is in the same range as the processor doing the whole multiplication itself, estimated at 0.1–0.2 ms. Whether the accelerator is actually faster than the processor can therefore only be decided by the board measurement of rung 4.

9.7 What limits rung 4, and what could come next

- **The writer** (75 cycles per output block in the compute phase): writing C in longer bursts or wider beats would let the compute phase run at the multiplier's pace of 60 cycles per block, about 1 500 cycles less.
- **Loading** (2 529 cycles, a quarter of the run): a second read port, so that A and B are loaded in parallel, would halve it.
- **The multiplier** (5 busy cycles per block pair): a pipelined multiplier that accepts a new pair every cycle is only worth building after the two points above.

10 Limitations

- The AXI memory model is ideal apart from its fixed write-response latency. The real path has a SmartConnect, a clock-domain crossing and DDR4 refresh, so its latency varies.
- The BRAM model reproduces the configured read latency of 1 cycle, not the Xilinx BRAM primitive itself.
- The address multipliers (`mult_gen`, constant ×4) are not in the testbench; the partitioner addresses the BRAM model directly. They have latency 0 in the design (checked in their `.xci` files), so they add no cycles.
- The AXI read model of rung 4 returns data at a fixed latency, one beat per cycle; the real HP port shares the DDR controller with the CPU.
- The CPU parts of the end-to-end times in Section 9 are estimates, not simulations.
- Nothing here covers timing closure; that needs the licensed implementation run for every rung.

11 Conclusion

Every part of the PL data path was simulated and gives the correct 50×50 result in all 96 simulation runs, including with the real HLS-generated result writer, under heavy back-pressure and after a restart without reset. One multiplication takes 101 186 cycles, or 1.012 ms at 100 MHz, and this is the PL compute time the board measurement should reproduce. The handshake monitors show exactly where that time goes: the partitioner needs 101 cycles per block pair, half of them wait states, while the multiplier works 5 of those cycles. The simulation also exposed a limit in the generated `RAM_INTERFACE`, a write-response wait after every row. It costs nothing today. For the first improvement I simulated two faster partitioners; the pipelined one (v5b) brings the PL run to 27 186 cycles, 3.7× faster, with all tests passing. It reaches that speed only if the memory confirms a write within 37 cycles, so the writer should be changed next. The remaining unknowns, the CPU's loading and read-back time and the interrupt, can only be answered on the board.

The improvement ladder (30 October) carries this further. Four rungs, each a separate git branch and each passing all tests, bring the PL run from 101 186 to 10 073 cycles (10.0×): a pipelined partitioner, a writer with overlapped write responses, a partitioner that copies A and B on chip once, and a burst data path in which the PL reads A and B from the processor's DDR. The last rung matters most for the end-to-end time, because it removes the CPU's 10 000 single BRAM writes; by estimate it brings one multiplication from about 3.3 ms to about 0.2 ms, which is close to what the processor needs on its own. The board measurements of the rungs, in order, will show how much of this holds in hardware.

A Reproducing the results

With GHDL (VHDL-2008, any back end) and Python 3 on the PATH:

```
cd /d C:\PLD\sim
python run_all.py
```

This compiles all sources, runs the 31 GHDL cases of the current design (about 12 minutes) and writes `results.json`; it ends with `ALL PASS`. The partitioner versions are run with `python run_variants.py` (or one at a time: `python run_variants.py v5b`); each is built in its own folder `build_<name>`. Single cases:

```
python run_perf.py hello 30        # data set, writer-model busy cycles
python run_rtl.py random 20 0 2    # data set, B_LATENCY, W_STALL_N, RUNS
```

Each prints its counters as JSON. The improvement ladder is run with

```
python run_ladder.py                    # all rungs
python run_ladder.py 3_partitioner_v6   # one rung (others from ladder_results.json)
python make_ladder_figures.py           # the two ladder figures
```

The HLS results are reproduced with `build_ip.bat` and `run_cosim.bat` in `C:\PLD\hls\RAM_INTERFACE` (no licence needed).

B Files

Path (under `C:\PLD\sim`)	**Content**
`tb_chain.vhd`, `run.py`	Original functional testbench (unchanged)
`tb_chain_perf.vhd`, `run_perf.py`	Performance testbench with handshake monitors
`tb_chain_rtl.vhd`, `run_rtl.py`	Testbench with the real `RAM_INTERFACE` RTL
`axi_mem_model.vhd`	AXI4 write-slave memory model
`hls_rtl\*.vhd`	Copy of the generated `RAM_INTERFACE` RTL (one `SIM PATCH` line)
`run_all.py`, `results.json`	Run all tests of the current design; their results
`variants\` `matrix_partitioner_B_v5a.vhd`, `_v5b.vhd`	The two faster partitioner versions (drop-in)
`run_variants.py`, `variants_results.json`	Run the tests per partitioner version; their results
`make_figures.py`	Draws the report figures from the two results files
`*.vhd` (design)	Copies of the IP sources used in the design (unchanged)
`variants\` `matrix_partitioner_B_v6.vhd`, `_v7.vhd`	Partitioners of ladder rungs 3 and 4
`steps\2_writer_fix\hls_rtl\*.vhd`	Generated RTL of the new writer (rung 2, one `SIM PATCH` line)
`axi_rd_mem_model.vhd`, `tb_chain_axi.vhd`	AXI4 read-slave model and testbench of rung 4
`ladder.json`, `run_ladder.py`	Definition of the rungs; runs them
`ladder_results.json`, `ladder.md`	Results of every rung; summary table
`make_ladder_figures.py`	Draws the two ladder figures